Software Requirements Engineering

Requirements Engineering Process – Analysis –Part I

Tibebe B.

Outline

- Overview
- Modeling techniques and principles
- Modeling languages
- Use case Modeling

Chapter objectives

- To understand the detail process and techniques in requirement analysis
- To develop practical skill of modeling and prioritizing requirements

Overview

- Recall that Analyzing requirements involves reaching a richer and more precise understanding of each requirement and representing sets of requirements in multiple ways.
- A **bulk of information** can be gathered through documentation, interviewing, questionnaires, observation and other tools such as prototyping and JAD about the system being studied.
- This information has to be **organized (structured**) somehow before giving it to designers who prepare technical specifications.

Cont…

- Thus, after elicitation the next step is to model the information with some graphical representation so that it will be easy to **undertsnd the current system** and to **visualize** the **proposed system** before the development.

- Visual requirements models can help you identify missing, extraneous, and inconsistent requirements.
- Business analysts might hope to find one technique that pulls everything together into a holistic depiction of a system's requirements.
  - Unfortunately, there is **no** such **all-encompassing diagram.**
- In fact, if you could model the entire system in a single diagram, that diagram would be just as **unusable**

Cont…

- An early goal of requirement analysis was to replace the classical functional specification with diagrams and notations that are more formal than narrative text.
- However, experience has shown that **analysis models should augment**—rather than replace—a requirements specification written in natural language.
- Developers and testers still benefit from the detail and precision that written requirements offer.

Cont…

- In structured systems analysis and design approach **separate models** are used to structure the data, logic and process aspects of the proposed system.
- Another effective modeling approach is an **object-oriented approach** to software development using class, sequence …diagrams
- Whether you use structural or object-oriented approaches modeling will help you to organize the information/requirement in such a way that it is easy for designers to understand.
  - Organizing and prioritizing requirements
  - Detecting conflicts and inconsistencies
  - Modeling (e.g., DFDs, UML diagrams)

Major tasks in Requirement analysis

- Analyzing the information received from users to **distinguish their task goals** from functional requirements, quality expectations, business rules, suggested solutions, and other information
  - Modeling
- **Decomposing high-level requirements** into an appropriate level of detail
- **Deriving functional requirements** from other requirements information
- Understanding the **relative** importance of **quality attributes**
- Negotiating **implementation priorities**
- Identifying **gaps in requirements or unnecessary requirements** as they relate to the defined scope

Modeling techniques and principles for requirement structuring and analysis

- A model is an abstraction, a representation of part of the real world.
  - Model is an idealization or abstraction of the actual process.
- An abstraction is often viewed as the process of stripping an idea or a system of its concrete or physical features.
- Hence modeling is **relevant abstraction and representation** of reality as well as mastering complexity.

Cont…

- A model represents **something that exists or is planned** in the real world and that in someway is too complex or large for us to understand it as it stands.
- Thus, a model is, in someway, simplified, or reduced in size, scope or scale.
- It helps to understand the system better. And it is the best way, scientific way to study reality.

Cont…

- Models of the **existing system (system as-is)** are used during requirements engineering.
  - clarify what the existing system does
  - as a basis for discussing its strengths and weaknesses.
- These then lead to requirements for the new system.

- Models of the **new system (system to-be)** are used to help explain the **proposed requirements** to other system stakeholders.
  - Engineers use these models to discuss **design proposals** and to document the system for implementation.

Cont…

- There are different visualizations of software modeling approaches.
- The first approach is based on **what to model**
- What to model?
    - Should we model every thing?
    - What part of the reality should be modeled?
        - In the context of software development?

Cont…

- Modeling could be mainly; process oriented, data oriented or object oriented
- Common Modeling in software engineering
  - Process Modeling
    - Logic Modeling
  - Data Modeling
  - Object Modeling

Cont…

- The other approach is based on the level of details to be represented.
- With this regard, there are three basic levels of modeling;
  - conceptual modeling (domain modeling)
  - logical modeling and
  - physical modeling
- While conceptual modeling is end-user oriented, logical modeling and Physical modeling are System oriented and Implementation oriented respectively.
- **Question**
  - Illustrate this approach taking a database design as example

Cont…

- In order to achieve the required objective with modeling it is important to adhere to the basic principles of modeling;
  - Best models are connected to reality
  - No single model is sufficient
  - Every model may be expressed at different precision
- Provide examples

Modeling tips

TABLE 12-1 Relating the customer's voice to analysis model components

Type of word	Examples	Analysis model components
Noun	People, organizations, software systems, data elements, or objects that exist	■ External entities, data stores, or data flow (DFD) ■ Actors (use case diagram) ■ Entities or their attributes (ERD) ■ Lanes (swimlane diagram) ■ Objects with states (STD)
Verb	Actions, things a user or system can do, or events that can take place	■ Processes (DFD) ■ Process steps (swimlane diagram) ■ Use cases (use case diagram) ■ Relationships (ERD) ■ Transitions (STD) ■ Activities (activity diagram) ■ Events (event-response table)
Conditional	Conditional logic statements, such as if/then	■ Decisions (decision tree, decision table, or activity diagram) ■ Branching (swimlane diagram or activity diagram)

Candidate artifacts for Requirement analysis

Table 1. Candidate artifacts for analysis modeling.

Artifact	Simple Tool	Description
Activity Diagram	Whiteboard	UML activity diagrams are used during analysis modeling to explore the logic of a usage scenario (see below), system use case, or the flow of logic of a business process. In many ways activity diagrams are the object-oriented equivalent of flow charts and data-flow diagrams (DFDs).
Class Diagram	Whiteboard	Class diagrams show the classes of the system, their inter-relationships (including inheritance, aggregation, and association), and the operations and attributes of the classes. During analysis you can use class diagrams to represent your conceptual model which depict your detailed understanding of the problem space for your system.
Constraint definition	Index card	A constraint is a restriction on the degree of freedom you have in providing a solution. Constraints are effectively global requirements for your project.
CRC model	Index cards	A Class Responsibility Collaborator (CRC) model is a collection of standard index cards, each of which have been divided into three sections, indicating the name of the class, the responsibilities of the class, and the collaborators of the class. Like class diagrams, CRC models are used during analysis modeling for conceptual modeling.
Data flow diagram (DFD)	Whiteboard	A data-flow diagram (DFD) shows the movement of data within a system between processes, entities, and data stores. When analysis modeling a DFD can be used to model the existing and/or proposed business processes that your system will support.
Entity/Relationship (E/R) diagram (data diagram)	Whiteboard	E/R diagrams show the main entities, their data attributes, and the relationships between those entities. Like class diagrams E/R diagrams can be used for conceptual modeling, in many ways E/R diagrams can be thought of as a subset of class diagrams.
Flow chart	Whiteboard	Flow charts are used in a similar manner to activity diagrams.
Robustness diagrams	Whiteboard	Robustness diagrams can be used to analyze usage scenarios to identify candidate classes and major user interface elements (screens, reports, ...).

Cont…

Sequence diagram	Whiteboard	Sequence diagrams are used to model the logic of usage scenarios. Sequence diagrams model the flow of logic within your system in a visual manner, enabling you to both explore and validate your logic.
State chart diagram	Whiteboard	State chart diagrams depict the various states, and the transitions between those states, that an entity exhibits. During analysis modeling state chart diagrams are used to explore the lifecycle of an entity that exhibits complex behavior.
System use case	Paper	A use case is a sequence of actions that provide a measurable value to an actor. A system use case includes high-level implementation decisions in it. For example, a system use case will refer to specific user interface components - such as screens, HTML pages, or reports. System use cases will also reflect fundamental architectural decisions, such as the use of ATMs versus cell phones to access your bank account.
UI prototype	Whiteboard	A user interface (UI) prototype models the user interface of your system. As a part of analysis modeling it enables you to explore the problem space that your system addresses in a manner that your project stakeholders understand. Remember, the user interface is the system to most people.
Usage scenario	Index card	A usage scenario is exactly what its name indicates - the description of a potential way that your system is used.
Use case diagram	Whiteboard	The use-case diagram depicts a collection of use cases, actors, their associations, and optionally a system boundary box. When analysis modeling a use case diagram can be used to depict the business functionality, at a high-level, that your system will support. It can also be used to depict the scope of the various releases of your system via the use of color or system boundary boxes.

System Perspectives

- **A functional perspective,** where you model the functional (from task perspective) behavior of the system
- A **structural perspective,** where you model the organization of a system or the structure of the data that is processed by the system
- A **behavioral perspective,** where you model the dynamic behavior of the system and how it responds to events
- An **interaction perspective,** where you model the interactions between a system and its environment, or between the components of a system.

Modeling Languages in Software Engineering

- Business Process Model and Notation (BPMN)
- **Structured-analysis modeling (Process oriented)**
- **Unified Modeling Language (UML)**
- SysML – System Modeling Language
- ArchiMate Modeling Language

Unified Modeling Language (UML)

- Standard language for object-oriented modeling.
- The UML has **13 diagram types** and so supports the creation of many different types of system model.
- Commonly used includes
  - **Use case diagrams,** which show the interactions between a system and its environment.
  - **Class diagrams,** which show the object classes in the system and the associations between these classes.
  - **Activity diagrams,** which show the activities involved in a process or in data processing.
  - **Sequence diagrams,** which show interactions between actors and the system and between system components.
  - **State diagrams,** which show how the system reacts to internal and external events.

What is UML?

- UML is a modeling language using a graphical notation for
  - Specifying, visualizing, constructing, and documenting software systems
  - Particularly useful for **OO design**
  - **Independent** of implementation language
- Unified Modeling Language
  - **From OMG Standard,** Object Management Group
  - Based on SE/ Systems development methodologies in early 90's
    - Booch, Jacobson, Yourden, Rumbaugh
  - UML v2.5.1 current version (released in 2017)

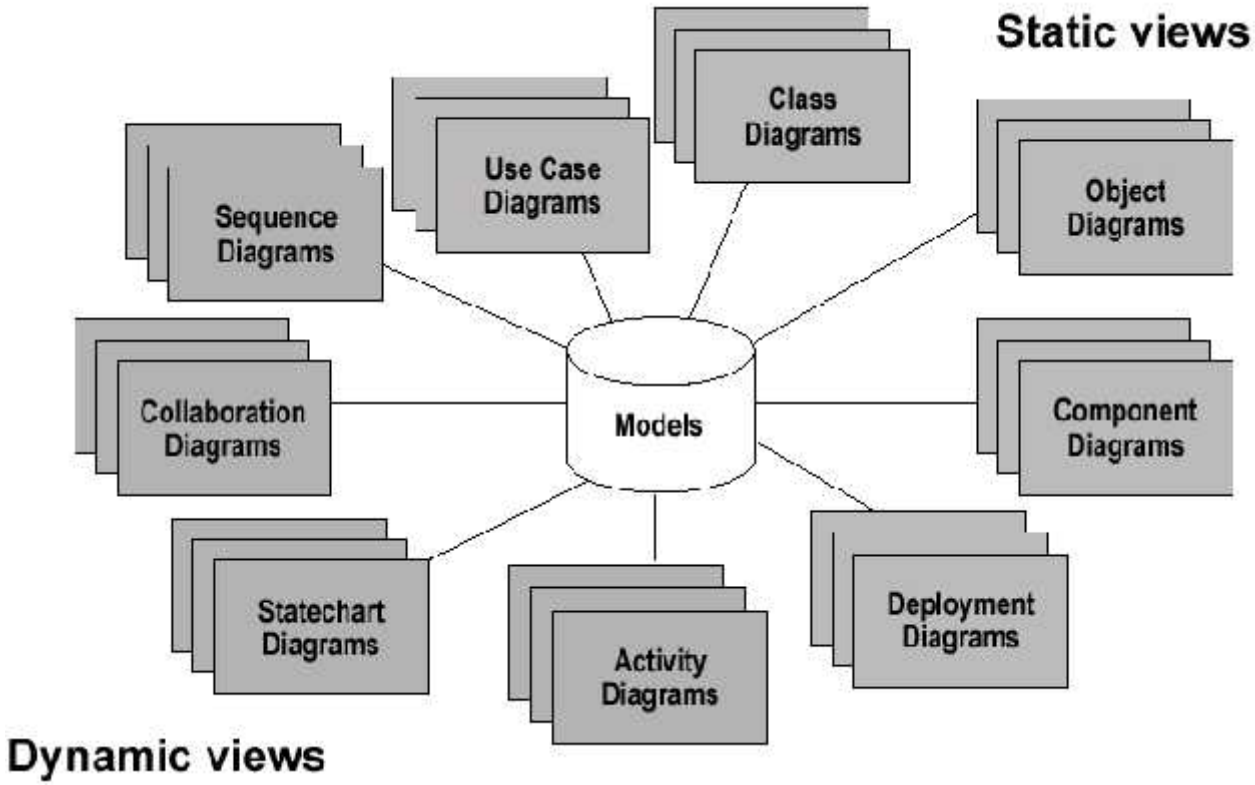
Why use UML

- It can be used from general initial design to very specific detailed design across the entire software development lifecycle
- Increase understanding/communication of product to customers and developers
- Support for diverse application areas
- **Support for UML in many software packages** today (e.g. Rational, Viso, plugins for popular IDE's like NetBeans, Eclipse, draw.io, Lucidchart)
- Based upon experience and needs of the user community

Cont…

- UML is an object-oriented modeling language (or more precisely, a collection of modeling languages) that is
  - **expressive**
  - **semi-formal**
  - capable of supporting incremental development
    - Elements can be hidden
    - Certain elements can be left incomplete
  - **process independent**
    - UML can be used with a variety software development process models
  - Customizable and extensible

Views and Diagrams in UML



Use Case Diagram

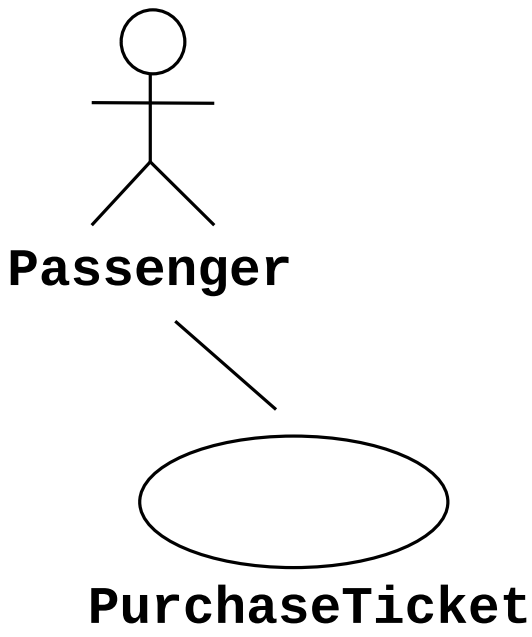
Scenario:

Online Library Management System

- An online library system allows users to Register and log in, Search for books, Borrow and return books, View borrowing history, Reserve a book if it is currently not available. It is also expected to help Librarians to Add, update, or remove books, View user borrowing history and Approve or deny reservations. The system should also allow Administrators to Create and manage librarian accounts and Generate reports.

- Task
  - **Draw a Use Case Diagram**

Use Case Diagram

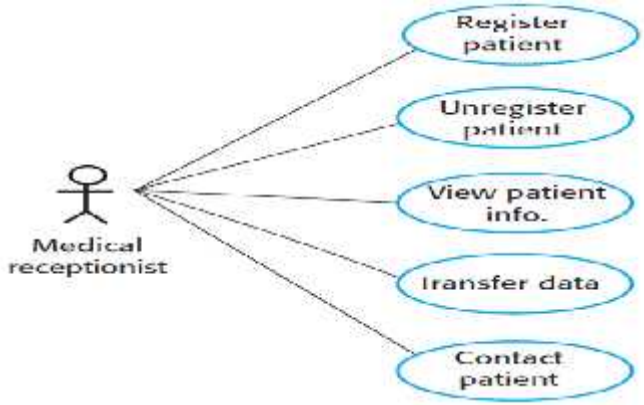


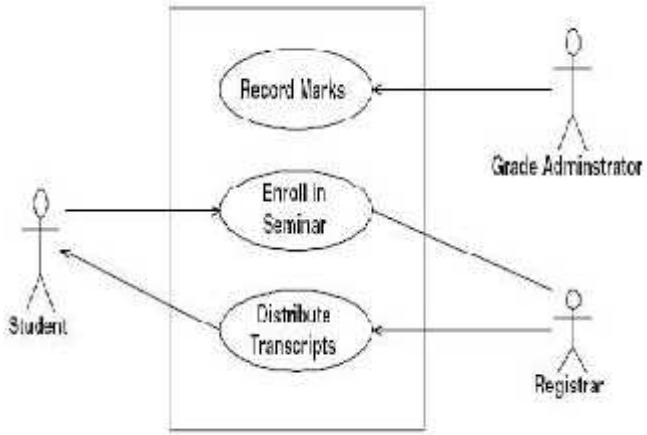
- Used during requirements **elicitation/analysis** to represent external behavior

- ***Actors*** represent **roles**, that is, a type of user of the system
- ***Use cases*** represent a **sequence of interaction** for a type of functionality; summary of scenarios
- The use case model is **the set of all use cases**.
- It is a complete description of the functionality of the system and its environment

Cont…

- Each use case represents a **discrete task** that involves external interaction with a System
- Use case diagrams give a simple overview of an interaction
- You need to add more detail for complete interaction description





Use Cases are useful in…

- Determining requirements
  - **New use cases often generate new requirements** as the system is analyzed and the design takes shape.
- Communicating with clients
  - Their notational simplicity makes use case diagrams a good way for developers to **communicate with clients**.
- Generating **test cases**
  - The collection of scenarios for a use case may suggest a suite of test cases for those scenarios.

Cont …

- You cannot successfully build a system if you do not know what it should do, and a critical aspect of this is exploring how people will actually use the system.
- Use case is an important technique for **modeling how people use** a system.
- Use case is the best-known approach for requirement elicitation/analysis
- Use case diagrams are one of the standard Unified Modeling Language (UML) artifacts

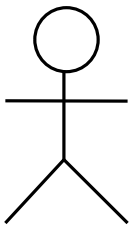
Essential use case vs system use case

- **Use Case:**
  - A use case describes something of value to an actor (often a person or organization).
- **Essential Use Case:**
  - An essential use case is a use case that is technology independent—it describes the fundamental business task without bringing technological issues into account.
  - Essential use cases are often used to explore usage-based requirements.
- **System Use Case:**
  - System use cases are use cases that bring technological concerns into account.
  - System use cases are the primary requirements artifact for the rational unified process (RUP).

Components of Use Case Model

- **Components of a use case model are:**
  - **Use cases.** A use case describes a sequence of actions that provide something of measurable value to an actor and is **drawn as a horizontal ellipse**.
  - **Actors.** An actor is a person, organization, or external system that plays a role in one or more interactions with your system. **Actors are drawn as stick figures**.
  - **Associations.** Associations describe interactions between actors and use cases and they are indicated with **solid lines**.
  - **System boundary boxes (optional).** The system boundary box is a rectangle around the use case used to indicate the scope of your system.

Actors



Passenger

- An actor models an **external entity** which communicates with the system:
  - User
  - External system
- An actor has a unique name and an optional description.
- Examples:
  - Passenger: A person in the train
  - GPS satellite: Provides the system with GPS coordinates

Identifying actors

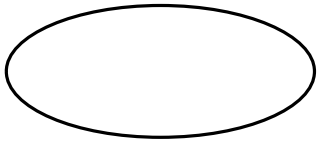
- An actor represents anything or anyone that interfaces with your system.
- The following questions can be asked to identify potential actors:
  - Who is the **main customer** of your system?
  - Who **obtains information** from this system?
  - Who **provides information** to the system?
  - Who **installs** and **operate** the system?
  - What **other systems interact** with this system?

Cont'd..

- Actors are role abstractions and do not necessarily directly map to persons.
- Actors do not model positions.
- Actors are usually named with singular nouns.
- Actors help you to identify the use cases they carry out.
- Once identified, actors can be described as follows:

Name: *Grade Administrator*
Description: *A Grade Administrator will input, update, and finalize in the system the marks that students receive on assignments, tests, and exams. Deans, professors (tenured, assistant, and associate), and teaching assistants are all potential grade administrators*

Use Case



PurchaseTicket

- A use case represents a class of functionality provided by the system as an event flow.

A use case **consists** (implicit or explicit) of:

- Unique name
- Participating actors
- Entry conditions
- Flow of events
- Exit conditions
- Special requirements

Identifying use cases

- One way to identifying use cases is to ask the following questions from the actors' point of view:
  - What are users in this role **trying to accomplish**?
  - To fulfill this role, what do users **need to be able to do**?
  - What are the **main tasks of** users in this role?
  - What information do users in this role **need to examine, create, or change**?
  - What do users in this role **need to be informed of** by the system?
  - What do users in this role **need to inform** the system about?

Cont'd..

- **For example**, from the point of view of the *Student* actor, you may discover that students:
  - Enroll in, attend, drop, fail, and pass seminars.
  - Need a list of available seminars.
  - Need to determine basic information about a seminar, such as its description and its prerequisites.
  - Obtain a copy of their transcript, their course schedules, and the fees due.
  - Pay fees, pay late charges, receive reimbursements for dropped and cancelled courses, receive grants, and receive student loans.
  - Graduate from school or drop out of it.
  - Need to be informed of changes in seminars, including room changes, time changes, and even cancellations.
  - Provide fundamental information about themselves such as their name, address, and phone number.

Cont…

- Another way to identify use cases is to ask stakeholders to brainstorm the various scenarios, often called usage scenarios, that the system may or may not support.

- **For example**, the following would be considered use case scenarios for a university information system:
  - A student wants to enroll in a seminar, but the registrar informs him that he does not have the prerequisites for it.
  - A student wanted to enroll in a seminar that she does have the prerequisites for and seats are still available in the seminar.

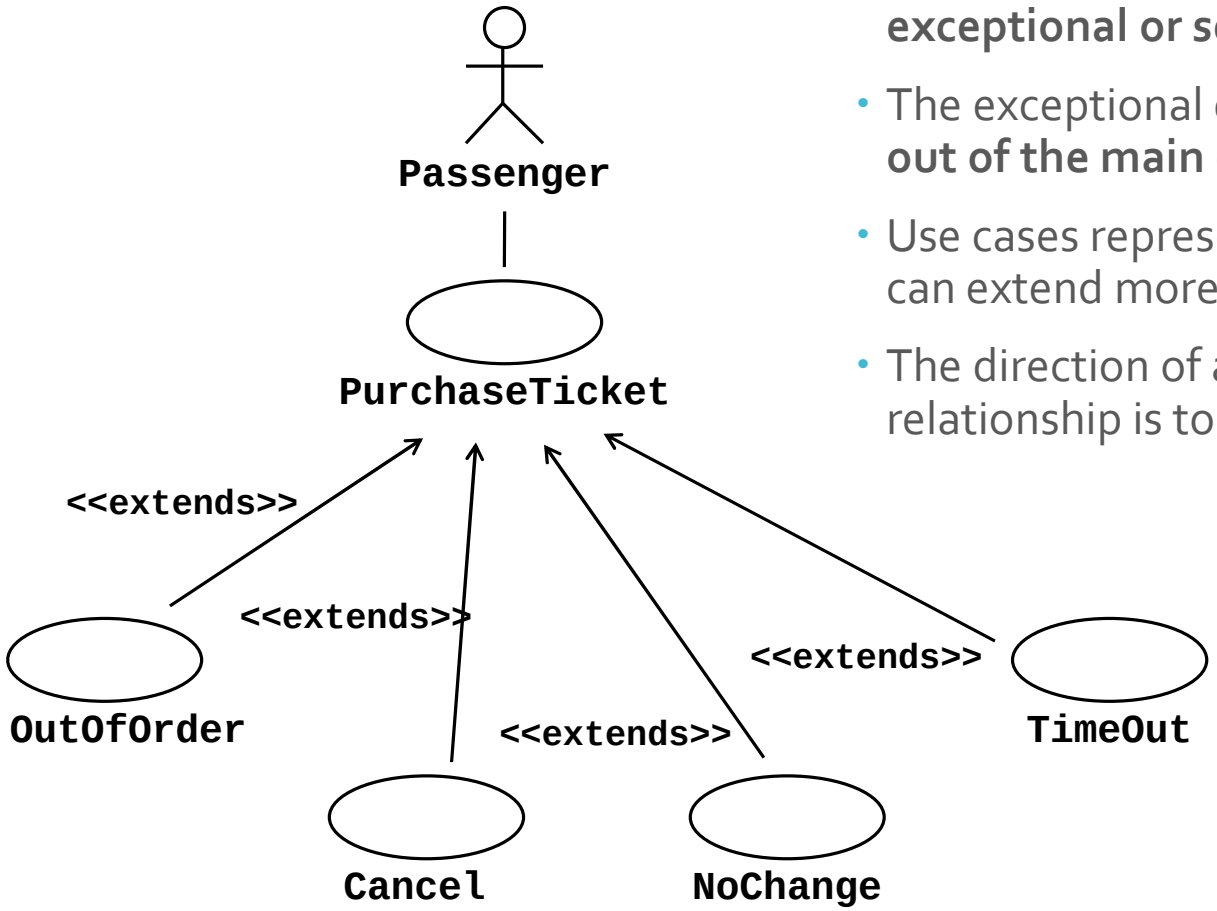
Identifying relationships

- Relationships among actors and use cases enable the developers and users to reduce the complexity of the model and increase its understandability.
- The relationships between actors and use cases are recognized when use cases are identified.
- **Communication (association) relationships:**
  - Is a relationship between actors and use cases and it represents the flow of information during the use case.

Identifying relationships cont'd..

- **Extend relationship**
  - Is a relationship where an **extending use case** continues the behavior of a **base use case**.
  - An extending use case continues the activity sequence of a base use case when the appropriate extension point is reached in the base use case and the extension condition is fulfilled
  - When the extending use case activity sequence is completed, the base use case continues.
  - **An extending use case is, effectively, an alternate course of the base use case.**
  - Use extend relationship for **exceptional, optional, or seldom-occurring behavior.**

The <<*extends*>> Relationship

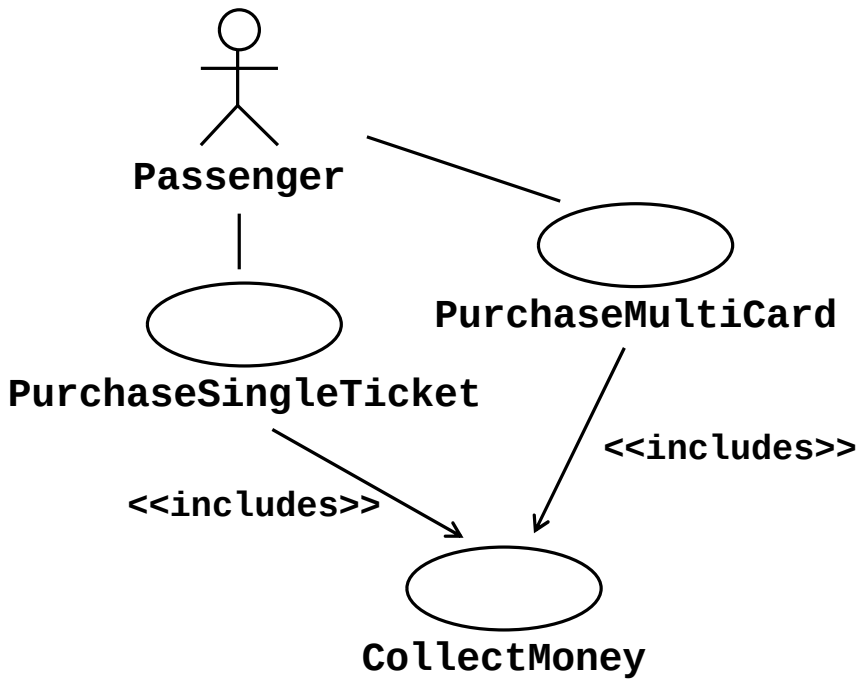


- <<extends>> relationships represent **exceptional or seldom invoked cases**.
- The exceptional event flows are **factored out of the main event flow** for clarity.
- Use cases representing exceptional flows can extend more than one use case.
- The direction of a <<extends>> relationship is to the extended use case

Identifying relationships cont'd..

- **Include (use) relationship**
  - Is a relationship denoting the **inclusion** of the behavior described by another use case.
  - The best way to think of an include association is that it is the invocation of a use case by another one.
  - Use include relationship for behavior that is shared across two or more use cases.

The <<*includes*>> Relationship



- <<includes>> relationship represents **behavior that is factored out of the use case**.
- <<includes>> behavior is factored out for reuse, not because it is an exception.
- The direction of a <<includes>> relationship is to the used use case (unlike <<extends>> relationships).

Identifying relationships cont'd..

- **Inheritance between actors**
  - An actor on a use case diagram can inherit from another actor.
  - When inheritance of an actor is used it should make sense to say the inheriting actor is or is like the inherited actor.
- **Inheritance between use cases**
  - Inheriting use case would completely replace one or more of the courses of action of the inherited use case.
  - Inheritance between use cases is not as common as the use of either extend or include, but it is still possible.

Cont…

- **Questions to be asked in determining the type of relationship are the following:**
  - Who is working/needing a given service/case? Use association/communication relationship
  - Is there any relationship that needs to be factored out among use cases? If yes 'include' will solve it
  - Is there any exceptional or optional cases? If yes "extend" will solve it
  - Are there two or more possibilities for accomplishing a case? If yes "generalization" will solve it

Describing a use case

- Description of a use case is part of the use case model.
- A use case would not be complete until it is complemented by a description.
- It is possible to adopt one of the several use case description styles/templates that are out there. (See the use case template adopted for this exercise in the following slide)

Use Case description : Example

Name: `Purchase ticket`

Participating actor: `Passenger`

Entry condition:

- `Passenger` standing in front of ticket distributor.
- `Passenger` has sufficient money to purchase ticket.

Event flow:

1. `Passenger` selects the number of zones to be traveled.
2. Distributor displays the amount due.
3. `Passenger` inserts money, of at least the amount due.
4. Distributor returns change.
5. Distributor issues ticket.

Exit condition:

- `Passenger` has ticket.

Cont…

- The essential elements of a use case:
  - A unique *identifier.*
  - A *name that states the user task, e.g. "Place an Order".*
  - A short *description.*
  - A list of *preconditions – conditions that must be satisfied or states in* which the system must be in before the use case may begin.
  - A list of *postconditions – conditions describing the state of the system* after the use case is *successfully completed.*
  - A *numbered list of steps that shows the sequence of dialog interactions* between actors and the system that leads from the preconditions to the postconditions.
  - a set of business rules applicable to the use case.

Use case description template

Section	Purpose
Name	The name of the use case. The name should implicitly express the user's intent or purpose of the use case.
Identifier [Optional]	A unique identifier used by other artifacts to reference this use case.
Description [Optional]	Several sentences summarizing the use case.
Goal [Optional]	The goal of this use case, i.e., the thing of measurable value that will be realized at its completion. If you cannot identify the goal you probably do not have a use case.
Preconditions [Optional]	A list of the conditions, if any, that must be met before a use case may be invoked.
Postconditions [Optional]	A list of the conditions, if any, that will be true once the use case finishes successfully.
Basic course of action	The main path of logic an actor follows through a use case. Often referred to as the "happy path," the "main success scenario," or the "main path" because it describes how the use case works when everything works as it normally should.
Alternate courses of action	The infrequently used path(s) of logic in a use case, paths that are the result of an alternate way to work, an exception, or an error condition. Sometimes called an extension.

Sample use case description

Name	Enroll in Seminar
Identifier	UC 17
Description	Enroll an existing student in a seminar for which she is eligible
Preconditions	The Student is registered at the University
Postconditions	The Student will be enrolled in the course she wants if she is eligible and room is available
Basic Course of Action	The use case begins when a student indicates they want to enroll in a seminar. The student inputs her name and student number into the system via UI23 Security Log-in Screen. The system verifies the student is eligible to enroll in seminars at the university according to business rule BR129 Determine Eligibility to Enroll. [Alt Course A] The system displays UI32 Seminar Selection Screen, which indicates the list of available seminars. The student indicates the seminar in which she wants to enroll. [Alt Course B: The Student Decides Not to Enroll] The system validates the student is eligible to enroll in the seminar according to the business rule BR130 Determine Student Eligibility to Enroll in a Seminar. [Alt Course C] The system validates the seminar fits into the existing schedule of the student according to the business rule BR143 Validate Student Seminar Schedule. The system calculates the fees for the seminar based on the fee published in the course catalog, applicable student fees, and applicable taxes. Apply business rules BR 180 Calculate Student Fees and BR45 Calculate Taxes for Seminar. The system displays the fees via UI33 Display Seminar Fees Screen. The system asks the student if she still wants to enroll in the seminar. The student indicates she wants to enroll in the seminar. The system enrolls the student in the seminar. The system informs the student the enrollment was successful via UI88 Seminar Enrollment Summary Screen. The system bills the student for the seminar, according to business rule BR100 Bill Student for Seminar. The system asks the student if she wants a printed statement of the enrollment. The student indicates she wants a printed statement. The system prints the enrollment statement UI89 Enrollment Summary Report. The use case ends when the student takes the printed statement.
Alternate Course A	The Student is Not Eligible to Enroll in Seminars A.3. The registrar determines the student is not eligible to enroll in seminars. A.4. The registrar informs the student he is not eligible to enroll. A.5. The use case ends.
Alternate Course B	The Student Decides Not to Enroll in an Available Seminar B.5. The student views the list of seminars and does not see one in which he wants to enroll. B.6. The use case ends
Alternate Course C	The Student Does Not Have the Prerequisites C.6. The registrar determines the student is not eligible to enroll in the seminar he chose. C.7. The registrar informs the student he does not have the prerequisites. C.8. The registrar informs the student of the prerequisites he needs.

Use case modeling tips

- Associations between actors and use cases **indicate the need for interfaces**
- Write use cases from the point of **view of the actor** in the active voice.
- Write scenario text, not functional requirements for easily understand the use cases
- A use case is neither **a class specification** nor a **data specification.**
- Organize your use case diagrams consistently.
- Remember the system responses to the actions of actors.
- Alternate courses of action are important.
- Do not get hung up on <<include>> and <<extend>> associations.
- Use cases drive user documentation.

In class Activity- 1

- A soccer club in the English Premier League (EPL) called the "OWLS" has season and priority tickets for its home matches (there are 20 teams in the EPL hence 19 home matches). Season tickets are applied for and issued prior to each season and provide admittance to every home game. Season tickets are available to existing season ticket holders from the previous season, then existing priority ticket holders in order of longevity (ie those who have been enrolled on the priority list the longest have the strongest claim). Priority tickets only are available to those who are registered as having a priority ticket entitlement. Tickets for a seat are identified by block, row and number. The price of a seat is dependent upon its block and row and is the same for all the fixtures in the season. All tickets are purchased by credit card. The fixtures are generated by the EPL in advance of the new season and provided to each participating club.
    - Identify possible actors and Use case (justify your selection)
    - Develop a use case diagram (Using tools like Visio)
    - Describe one of the use cases

Review questions

- Explain the three basic principles of modeling with example?
- Explain why requirement structuring is required?
- Explain modeling as a thinking tool?
- What aspect of a system do we model?
- How useful is a use case diagram?
- What are the major components of a use case model ?

Thank you